



University
of Bremen



Deutsches Zentrum
für Luft- und Raumfahrt
German Aerospace Center

UNIVERSITÄT BREMEN

PROJECT REPORT 2024

Smart Sensor Fusion Robust Trajectory Initialization and Estimation

Presented by

Chawda, Vimalkumar (6256031)

Supervised by

Dr. Stephan THEIL

September 17, 2025

Contents

1	Introduction	6
1.1	Motivation	6
1.1.1	The Demand for Resilient Navigation	6
1.1.2	Complementary Strengths of GNSS and IMU Sensors	7
1.2	Problem Statement	7
1.2.1	Static Initial Alignment as Wahba’s Problem	7
1.2.2	Impact on Downstream Navigation	8
1.3	Contributions	8
1.4	Report Outline	8
2	Background and Mathematical Models	10
2.1	Assumptions	10
2.1.1	Sensor Suite and Synchronization	10
2.1.2	Generic Assumptions	10
2.1.3	Implementation-Specific Details	11
2.2	Coordinate Systems	11
2.3	Gravity and Earth-Rotation Models	11
2.3.1	Normal Gravity	11
2.3.2	Earth Rotation	12
2.4	Attitude Determination — Wahba’s Problem	12
2.4.1	Observability and Vector Set	12
2.4.2	Closed-Form Solutions	12
2.4.3	Implementation Notes (Python)	12
2.5	Extended Kalman Filter for GNSS/IMU Fusion	13
2.5.1	System Model	13
2.5.2	Measurement Model	13
2.5.3	State Correction and Reset	13
3	Methodology and Implementation	14
3.1	Numerical Workflow for Initial Attitude Determination	14
3.1.1	Input Vector Preparation	14
3.1.2	Solving Wahba’s Problem	15

3.1.3	Robustness and Solver Selection	15
3.2	Performance Evaluation of Attitude Initialization Methods	16
3.2.1	Evaluation Metrics	16
3.2.2	Experimental Setup	16
3.2.3	Results and Discussion	16
3.3	Representation and Validation of Attitude	16
3.3.1	Euler Angles and Quaternions	16
3.3.2	Residual Alignment Error and Acceptance	16
3.4	Python Implementation Snippets	16
4	Experimental Setup and Data	20
4.1	Dataset	20
4.1.1	Overview	20
4.1.2	Sensor Suite, Timing, and Acquisition	20
4.2	File Formats and Data Ingestion	21
4.3	Statistical Properties and Noise Models	21
4.4	Ground-Truth Considerations	21
4.5	Execution Pipeline and Reproducibility (Python)	21
4.5.1	Software Architecture and Invocation	22
4.5.2	Computational Resources	22
4.5.3	Artifact Generation	23
4.5.4	Reproducibility Checklist (Python)	23
5	Results and Analysis	24
5.1	Initial Attitude Determination Comparison	24
5.2	Integrated Trajectory Analysis (Dead-Reckoning)	25
5.3	Kalman Filter Fusion Performance	25
5.4	Overall Performance Summary	26
5.5	Visual Analysis of Sensor and Filter Data	26
5.5.1	Initial Location and Derived Sensor Data	26
6	Conclusion	32
6.1	Conclusion	32
7	User Manual for the IMU/GNSS Pipeline	36
7.1	Where to Download	36
7.2	System Requirements	36
7.3	Installation	37
7.3.1	Python Environment	37
7.4	Repository Layout	37
7.5	Quick Start (How to Run)	37
7.5.1	Python — One-Command Demo	37

- 7.6 User-Facing Entry Points (Python) 38
- 7.7 Inputs Required 38
- 7.8 What the Pipeline Does (Task Summary) 38
- 7.9 Outputs 39
- 7.10 Changing Parameters and Inputs 39
 - 7.10.1 Choose Datasets 39
 - 7.10.2 Sensor Rates and Filter Tuning 39
 - 7.10.3 Example: Config-Driven Runs 39
- 7.11 Troubleshooting 39
- 7.12 Reproducibility Checklist 40

List of Figures

4.1	Automated execution pipeline invoked by the Python runners in <code>PYTHON/src</code> .	22
5.1	Pure INS (dead-reckoned) trajectory vs. GNSS in ECEF. The unaided INS diverges rapidly due to noise and biases (double integration of accelerometer errors). Rapid divergence in this figure highlights why MEMS-based INS systems require GNSS updates for sustained accuracy.	25
5.2	Derived and Measured GNSS and IMU data	27
5.3	Fused trajectory (TRIAD initialization) in ECEF.	28
5.4	Initial location derived from GNSS coordinates (full page width for maximum readability).	28
5.5	Overlay of fused IMU+GNSS navigation state with ground truth	29
5.6	Quaternion comparison between EKF-fused solution and reference (full page width). Small differences in attitude over time can point to misalignment, lever arm, or time base issues.	30
5.7	Differences between fused IMU+GNSS navigation state and ground truth . .	31

List of Tables

1.1	Qualitative comparison of GNSS and IMU error characteristics [2]	7
2.1	Summary of assumptions (generic and implementation-specific). [2]	11
4.1	Nominal noise statistics used for EKF tuning. [2]	21
4.2	Representative end-to-end execution timing (Python).	23
5.1	Vector misalignment after initial attitude determination.	24
5.2	Kalman filter performance metrics for all methods (Task 7.5). RMS: root mean square error over the run; End: final error at last epoch. Residuals are the innovation (GNSS-INS difference) at GNSS update times. Across valid datasets, RMS position errors remain at the decimetre level and residuals are consistent with the assumed GNSS noise model (~ 0.5 m).	26
6.1	Summary of EKF performance (Task 7.5): RMSE and end error for each method and dataset.	33

Chapter 1

Introduction

This report details the design, implementation, and analysis of a software system developed to fulfill the requirements of the "Master Project: Postprocessing of IMU and GNSS Data." This chapter establishes the context and rationale for the research, presents a rigorous formulation of the initial-alignment problem that motivates the work, enumerates the specific scientific and engineering contributions made, and provides a detailed roadmap for the remainder of the report. The reader is assumed to have a basic familiarity with inertial navigation principles. [2]

1.1 Motivation

1.1.1 The Demand for Resilient Navigation

Over the past decade, the proliferation of autonomous and semi-autonomous systems has shifted reliable navigation from a convenience to a safety-critical requirement. [2] Road vehicles must maintain lane-level accuracy in urban canyons; unmanned aerial vehicles (UAVs) must operate safely beyond visual line-of-sight; and wearable and handheld devices must offer seamless indoor-outdoor positioning for emergency services and augmented-reality (AR) applications. [2] These scenarios share three common traits:

- (i) **Stringent accuracy targets.** Lane keeping in dense traffic requires sub-metre errors; precision agriculture and survey operations demand centimetre-level repeatability. [2]
- (ii) **Continuous availability.** Position and attitude outputs must be delivered at fixed, low-latency intervals (50 Hz to 200 Hz for robotic control loops) regardless of environmental conditions. [2]
- (iii) **Robustness to interference and outages.** GNSS signals are susceptible to masking, spoofing, or unintentional jamming. [2]

1.1.2 Complementary Strengths of GNSS and IMU Sensors

GNSS receivers observe pseudo-ranges and carrier Doppler shifts from multiple satellites, yielding globally referenced position and velocity estimates whose long-term error statistics are dominated by *white* measurement noise. [2] In contrast, a strapdown IMU measures specific force and angular rate at hundreds to thousands of samples per second, enabling smooth interpolation between GNSS epochs and high-rate control actuation. [2] However, IMU outputs are corrupted by biases whose effects grow *unbounded* once integrated. [2] Table 1.1 contrasts the two sensor classes.

Table 1.1: Qualitative comparison of GNSS and IMU error characteristics [2]

Characteristic	GNSS	IMU
Update rate	1 Hz to 20 Hz (consumer)	100 Hz to 2000 Hz (MEMS)
Short-term noise	0,5 m RMS (code)	$\sim 1 \times 10^{-3}$ (m/s ²) RMS
Long-term stability	Unbiased	Bias drift (10 μ g – 100 mg)
Blockage sensitivity	High	None
Absolute reference	Yes	No

The complementary nature of these error sources has driven the development of **loose**, **tight**, and **deeply coupled** integration architectures. [2] Regardless of coupling depth, a reliable *initial attitude* is essential because the inertial mechanisation algorithm propagates errors in orientation directly into velocity and position estimates. [2] The work presented here therefore focuses on high-accuracy *static alignment* prior to motion. [2]

1.2 Problem Statement

1.2.1 Static Initial Alignment as Wahba's Problem

Consider an IMU rigidly mounted on a platform at an unknown orientation relative to a local NED frame $\mathcal{F}_n$. [2] During a stationary period two deterministic reference vectors are available:

- (a) The local gravity vector $\mathbf{g}^n = [0 \ 0 \ g]^\top$ with $g \approx 9,81$ m/s². [2]
- (b) The projection of the Earth's angular velocity $\boldsymbol{\omega}_{ie}^n$. [2]

The IMU measures their counterparts $\mathbf{g}^b$ and $\boldsymbol{\omega}_{ie}^b$ in the body frame $\mathcal{F}_b$. [2] The goal is to estimate the rotation matrix $\mathbf{C}_b^n$ that satisfies: [5]

$$\mathbf{v}_k^n = \mathbf{C}_b^n \mathbf{v}_k^b + \varepsilon_k, \quad k \in \{1,2\}, \quad (1.1)$$

where ε_k captures sensor noise. [5] With two non-collinear vectors the attitude is fully observable. [5] This formulation maps directly onto Wahba's minimisation problem; the practical challenge lies in solving for $\mathbf{C}_b^n$ with milliradian-level accuracy using noisy MEMS sensors while keeping computational overhead suitable for embedded processors. [5]

1.2.2 Impact on Downstream Navigation

Any systematic error $\delta\theta$ in the initial rotation induces first-order position drift of magnitude $g t^2 \|\delta\theta_h\|$ in the horizontal plane and $\|\omega_{ie}\| t^2 \|\delta\theta_v\|$ in the vertical channel, where t is elapsed time and the subscripts denote horizontal/vertical components. [2] A 1° misalignment therefore produces 15 m of cross-track error after 60 s. [2] Reducing initial attitude error below 0.1° is thus critical for navigation systems targeting metre-level accuracy. [2]

1.3 Contributions

This project makes the following original contributions in fulfillment of the specified mandate:

- C1.** The development of a complete, open-source post-processing framework in **Python (3.10+)**, providing a modular and extensible toolchain for parsing, synchronizing, aligning, and fusing IMU and GNSS data from raw sensor files. [2]
- C2.** A systematic implementation and quantitative benchmarking of three classical solutions to Wahba's problem—TRIAD, Davenport's Q-Method, and Singular Value Decomposition (SVD)—for the task of static initial attitude alignment. [5]
- C3.** The design and implementation of a robust algorithm for resolving the **integer-second ambiguity** between PPS-driven IMU timestamps and the absolute GPS Second of Week provided in the GNSS data stream, a critical prerequisite for accurate data fusion. [2]
- C4.** The design and implementation of a robust **lift-off detection** algorithm based on sliding-window thresholding of the accelerometer signal magnitude, enabling the automatic transition from static alignment to in-flight filtering. [2]
- C5.** The development of a comprehensive **unit testing suite** using the **pytest** framework. This suite provides automated verification and validation for all key software modules, ensuring code correctness, robustness, and adherence to modern software engineering practices as required by the project mandate. [2]

1.4 Report Outline

For clarity the document is structured as follows:

provides a rigorous treatment of coordinate transformations, Earth-gravity and rotation models, Wahba's problem formulations, and the discrete-time extended Kalman filter used for data fusion. [2]

details the methods for determining the initial attitude from a stationary start, and the Extended Kalman Filter used to fuse the IMU and GNSS data during flight. [2]

describes the software architecture and specific Python implementation choices, including data ingestion, ambiguity resolution, lift-off detection, and the unit testing framework. [2]

presents side-by-side comparisons of attitude-estimation errors, inertial mechanization drift rates, and EKF fused-solution accuracy, supported by figures and statistical tables. [2]

summarises the key findings, outlines limitations, and proposes extensions such as tightly coupled carrier-phase updates and magnetometer-augmented alignment. [2]

Chapter 2

Background and Mathematical Models

This chapter outlines the theoretical underpinnings and mathematical models used in the project. It begins by stating the key assumptions, then defines the coordinate systems and environmental models (gravity, Earth rotation) necessary for inertial navigation. The chapter proceeds to formulate the initial attitude determination as an instance of Wahba’s problem and details the classical methods used for its solution. Finally, it describes the 15-state error-state Extended Kalman Filter (EKF) designed for the loosely-coupled fusion of IMU and GNSS data. [2]

2.1 Assumptions

2.1.1 Sensor Suite and Synchronization

Due to confidentiality constraints, specifics of the sensor suite (IMU, GNSS, and auxiliary sensors), time-tagging, and synchronization are intentionally omitted. The analyses remain valid and are presented without hardware identifiers.

2.1.2 Generic Assumptions

- **Stationary IMU during Alignment.** *During the initial alignment interval, the IMU is assumed to experience no linear acceleration; thus, the accelerometers measure the specific force due solely to gravity. This enables bias estimation via static averaging and allows Zero-Velocity Updates (ZUPT) in validation.* [2]
- **Magnetometer Excluded.** *The attitude-initialisation process relies exclusively on accelerometers and gyroscopes; magnetometers are not used.* [2]
- **Small Gyro Biases.** *Gyroscope biases are assumed small over the alignment window or mitigated by averaging; residual biases are handled in the EKF.* [2]

2.1.3 Implementation-Specific Details

Implementation specifics are intentionally omitted due to confidentiality constraints.

Table 2.1: Summary of assumptions (generic and implementation-specific). [2]

Assumption	Description
Generic Assumptions	
<i>IMU is stationary</i>	<i>No linear acceleration during alignment; accelerometers measure only gravity.</i>
<i>No magnetometer</i>	<i>Purely inertial alignment using accelerometers and gyroscopes.</i>
<i>Gyro biases are small</i>	<i>Removed/mitigated by averaging; residuals estimated within EKF.</i>
Implementation-Specific Details	
<i>Implementation-specific items</i>	<i>Intentionally omitted due to confidentiality constraints.</i>

2.2 Coordinate Systems

Four reference frames are employed throughout this work: [2]

- **ECEF (e-frame):** Earth-Centred, Earth-Fixed Cartesian frame; origin at Earth's geocentre. *GNSS inputs in this project are provided in ECEF.*
- **Geodetic:** Curvilinear coordinates latitude (ϕ), longitude (λ), and ellipsoidal height (h) on WGS-84.
- **NED (n-frame):** Local tangent frame with axes North, East, Down.
- **Body (b-frame):** Sensor-fixed frame attached to the IMU.

We denote by $\mathbf{C}_y^x$ the DCM that maps vector components from frame y to frame x . The transformation from n to e is [2]:

$$\mathbf{C}_n^e(\phi, \lambda) = \begin{bmatrix} -\sin \phi \cos \lambda & -\sin \phi \sin \lambda & \cos \phi \\ -\sin \lambda & \cos \lambda & 0 \\ -\cos \phi \cos \lambda & -\cos \phi \sin \lambda & -\sin \phi \end{bmatrix}, \quad \mathbf{C}_e^n = (\mathbf{C}_n^e)^\top. \quad (2.1)$$

2.3 Gravity and Earth-Rotation Models

2.3.1 Normal Gravity

The WGS-84 Somigliana model gives the surface gravity magnitude as a function of latitude ϕ [2]:

$$g(\phi) = \frac{9.7803253359(1 + 0.00193185265241 \sin^2 \phi)}{\sqrt{1 - 0.00669437999013 \sin^2 \phi}} \quad [\text{m/s}^2]. \quad (2.2)$$

In the NED frame: $\mathbf{g}^n = [0 \ 0 \ g(\phi)]^\top$. For $\phi = -31.87^\circ$, $g \approx 9.794 \text{ m/s}^2$. [2]

2.3.2 Earth Rotation

The Earth rotation rate is $\omega_E \approx 7.292115 \times 10^{-5} \text{ rad/s}$. In ECEF: $\boldsymbol{\omega}_{ie}^e = [0 \ 0 \ \omega_E]^\top$. Transforming to NED yields [2]:

$$\boldsymbol{\omega}_{ie}^n = \mathbf{C}_e^n \boldsymbol{\omega}_{ie}^e = \begin{bmatrix} \omega_E \cos \phi \\ 0 \\ -\omega_E \sin \phi \end{bmatrix}. \quad (2.3)$$

Its horizontal component $\omega_E \cos \phi$ underpins gyro-compassing for heading. [2]

2.4 Attitude Determination — Wahba's Problem

Given N weighted vector pairs $\{a_k, \mathbf{v}_k^b, \mathbf{v}_k^n\}$, Wahba's problem seeks $\mathbf{C}_b^n \in SO(3)$ minimising [5]:

$$L(\mathbf{C}_b^n) = \frac{1}{2} \sum_{k=1}^N a_k \left\| \mathbf{v}_k^n - \mathbf{C}_b^n \mathbf{v}_k^b \right\|^2. \quad (2.4)$$

2.4.1 Observability and Vector Set

With two non-collinear references the attitude is uniquely observable. We use $\mathbf{v}_1^n = \mathbf{g}^n$, $\mathbf{v}_2^n = \boldsymbol{\omega}_{ie}^n$ and their body counterparts $\mathbf{v}_1^b = \mathbf{g}^b$, $\mathbf{v}_2^b = \boldsymbol{\omega}_{ie}^b$, with weights a_1, a_2 typically chosen to reflect SNR (often $a_1 \geq a_2$). [5]

2.4.2 Closed-Form Solutions

Three classical closed-form solutions are used [3, 1, 4]:

1. **TRIAD:** Builds orthonormal triads in each frame and aligns them; fast and simple, but prioritises the first vector. [3]
2. **Davenport's Q-method:** Forms the 4×4 symmetric matrix $\mathbf{K}$ and takes the principal eigenvector as the attitude quaternion; numerically robust. [1]
3. **SVD/Procrustes:** Compute $\mathbf{B} = \sum a_k \mathbf{v}_k^n (\mathbf{v}_k^b)^\top = \mathbf{U} \boldsymbol{\Sigma} \mathbf{V}^\top$; set $\mathbf{C}_b^n = \mathbf{U} \text{diag}(1, 1, \det(\mathbf{U}) \det(\mathbf{V})) \mathbf{V}^\top$ to enforce $\det(\mathbf{C}_b^n) = +1$. [4]

2.4.3 Implementation Notes (Python)

In the provided repository, the three solvers share a unified interface. Body-frame vectors are obtained by averaging the first N_{stat} IMU samples; navigation references are constructed from geodetic latitude (derived from GNSS ECEF) using (2.2) and (2.3). Outputs are expressed as DCM, Euler angles, and unit quaternions for subsequent use in EKF initialisation.

2.5 Extended Kalman Filter for GNSS/IMU Fusion

We employ a loosely-coupled, 15-state error-state EKF that estimates navigation errors and sensor biases. The error state is [2]:

$$\delta \mathbf{x} = \begin{bmatrix} \delta \mathbf{p}^{n\top} & \delta \mathbf{v}^{n\top} & \boldsymbol{\theta}^\top & \delta \mathbf{b}_a^\top & \delta \mathbf{b}_g^\top \end{bmatrix}^\top. \quad (2.5)$$

2.5.1 System Model

Linearised continuous-time dynamics:

$$\dot{\delta \mathbf{p}}^n = -[\boldsymbol{\omega}_{en}^n \times] \delta \mathbf{p}^n + \delta \mathbf{v}^n, \quad (2.6)$$

$$\dot{\delta \mathbf{v}}^n = -(2[\boldsymbol{\omega}_{ie}^n \times] + [\boldsymbol{\omega}_{en}^n \times]) \delta \mathbf{v}^n + [\mathbf{f}^n \times] \boldsymbol{\theta} + \mathbf{C}_b^n \delta \mathbf{b}_a + \delta \mathbf{g}^n, \quad (2.7)$$

$$\dot{\boldsymbol{\theta}} = -[\boldsymbol{\omega}_{in}^n \times] \boldsymbol{\theta} - \mathbf{C}_b^n \delta \mathbf{b}_g, \quad (2.8)$$

$$\dot{\delta \mathbf{b}}_a = -\frac{1}{\tau_a} \delta \mathbf{b}_a + \mathbf{w}_{ba}, \quad \dot{\delta \mathbf{b}}_g = -\frac{1}{\tau_g} \delta \mathbf{b}_g + \mathbf{w}_{bg}. \quad (2.9)$$

Biases follow first-order Gauss–Markov processes with time constants τ_a, τ_g . Discretisation yields $\boldsymbol{\Phi}_k \approx \mathbf{I} + \mathbf{F} \Delta t$ and process noise $\mathbf{Q}_k$ via standard continuous-to-discrete mapping.

Prediction:

$$\delta \hat{\mathbf{x}}_k^- = \mathbf{0}, \quad (2.10)$$

$$\mathbf{P}_k^- = \boldsymbol{\Phi}_{k-1} \mathbf{P}_{k-1}^+ \boldsymbol{\Phi}_{k-1}^\top + \mathbf{Q}_{k-1}. \quad (2.11)$$

2.5.2 Measurement Model

We use position and velocity from GNSS (converted from ECEF to NED) against INS propagation:

$$\mathbf{z}_k = \begin{bmatrix} \hat{\mathbf{p}}_{\text{INS}}^n - \mathbf{p}_{\text{GNSS}}^n \\ \hat{\mathbf{v}}_{\text{INS}}^n - \mathbf{v}_{\text{GNSS}}^n \end{bmatrix}, \quad \mathbf{H}_k = \begin{bmatrix} \mathbf{I}_3 & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{I}_3 & \mathbf{0} & \mathbf{0} & \mathbf{0} \end{bmatrix}. \quad (2.12)$$

Update:

$$\mathbf{K}_k = \mathbf{P}_k^- \mathbf{H}_k^\top (\mathbf{H}_k \mathbf{P}_k^- \mathbf{H}_k^\top + \mathbf{R}_k)^{-1}, \quad (2.13)$$

$$\delta \hat{\mathbf{x}}_k^+ = \mathbf{K}_k \mathbf{z}_k, \quad (2.14)$$

$$\mathbf{P}_k^+ = (\mathbf{I} - \mathbf{K}_k \mathbf{H}_k) \mathbf{P}_k^-. \quad (2.15)$$

2.5.3 State Correction and Reset

The estimated errors correct the navigation state:

$$\hat{\mathbf{p}}^n \leftarrow \hat{\mathbf{p}}^n - \delta \hat{\mathbf{p}}^n, \quad \hat{\mathbf{v}}^n \leftarrow \hat{\mathbf{v}}^n - \delta \hat{\mathbf{v}}^n, \quad (2.16)$$

$$\hat{\mathbf{C}}_b^n \leftarrow (\mathbf{I} - [\hat{\boldsymbol{\theta}} \times]) \hat{\mathbf{C}}_b^n, \quad \hat{\mathbf{b}}_a \leftarrow \hat{\mathbf{b}}_a + \delta \hat{\mathbf{b}}_a, \quad \hat{\mathbf{b}}_g \leftarrow \hat{\mathbf{b}}_g + \delta \hat{\mathbf{b}}_g. \quad (2.17)$$

Finally, reset $\delta \hat{\mathbf{x}}_k^+ \leftarrow \mathbf{0}$ before the next prediction. [2]

Chapter 3

Methodology and Implementation

This chapter translates the theoretical models established in Chapter 2 into an operational software pipeline written in **Python (3.10+)**. [2] It details the numerical workflow for each of the core project tasks, from initial attitude determination to the performance evaluation of the implemented algorithms. The chapter is structured to first describe the end-to-end workflow for attitude estimation, then detail the implementation of the core solvers, present a quantitative performance evaluation, and finally discuss the methods for attitude representation and validation. [2]

Repository and entry points. All scripts referenced in this chapter reside under `PYTHON/src` in the repository `Smart_Sensor_Fusion_for_Robust_initialization_and_Accurate_Trajectory_Estimation`. Typical runners include:

```
python PYTHON/src/run_triad_only.py --run-id X002
python PYTHON/src/run_all_methods.py --run-id X002
```

Outputs (plots, `.mat/.npz`, CSV metrics) are saved in `PYTHON/results/`. GNSS inputs are ECEF-only and are converted internally to geodetic / NED as needed.

3.1 Numerical Workflow for Initial Attitude Determination

The first critical step in the post-processing chain is the estimation of the initial attitude, represented by the direction cosine matrix (DCM) $\mathbf{C}_b^n$, which aligns the IMU body frame (b) with the local navigation frame (n). [5] This procedure solves Wahba's problem (see Section 2.4) by comparing inertial vectors measured by the IMU against known reference vectors derived from GNSS data.

3.1.1 Input Vector Preparation

Two vector pairs are constructed: reference vectors in the navigation frame and their counterparts in the body frame. [2]

- **Reference vectors in n -frame:** The initial geodetic latitude ϕ and longitude λ are obtained by converting the first valid GNSS ECEF fix from `GNSS_X002.csv`. (*GNSS files contain only ECEF; no lat/lon/alt are stored.*) Using ϕ , the normal gravity magnitude $g(\phi)$ is computed via Somigliana (Eq. 2.2) and the Earth-rate vector ω_{ie}^n via Eq. 2.3. Reference vectors are $\mathbf{g}^n$ and ω_{ie}^n .
- **Measured vectors in b -frame:** The first 10 seconds of IMU data (e.g., $N_{\text{stat}}=4000$ samples at 400 Hz from `IMU_X003.dat`) are averaged to obtain $\mathbf{g}^b$ (accelerometers) and ω_{ie}^b (gyroscopes). Static averaging implicitly estimates and mitigates biases over the alignment window.

3.1.2 Solving Wahba's Problem

Three closed-form methods are implemented in Python to solve Wahba's problem with a unified interface: TRIAD [3], Davenport's $\mathcal{Q}$ -method [1], and SVD/Procrustes [4]. This allows both flexibility and direct comparison.

TRIAD Method

TRIAD constructs orthonormal triads from the two vector pairs in each frame, then aligns them:

$$\mathbf{t}_1^n = \frac{\mathbf{v}_1^n}{\|\mathbf{v}_1^n\|}, \quad \mathbf{t}_2^n = \frac{\mathbf{v}_1^n \times \mathbf{v}_2^n}{\|\mathbf{v}_1^n \times \mathbf{v}_2^n\|}, \quad \mathbf{t}_3^n = \mathbf{t}_1^n \times \mathbf{t}_2^n,$$

(and analogously for the b -frame). The attitude is $\mathbf{C}_b^n = [\mathbf{t}_1^n \ \mathbf{t}_2^n \ \mathbf{t}_3^n] [\mathbf{t}_1^b \ \mathbf{t}_2^b \ \mathbf{t}_3^b]^\top$. [3]

Davenport's $\mathcal{Q}$ -method

Davenport's method builds a symmetric 4×4 matrix $\mathbf{K}$ from weighted pairs and returns the principal eigenvector as the optimal quaternion; $\mathbf{C}_b^n$ follows from the quaternion. [1]

SVD / Procrustes

With weights w_k , form $\mathbf{B} = \sum_k w_k \mathbf{v}_k^n (\mathbf{v}_k^b)^\top = \mathbf{U} \Sigma \mathbf{V}^\top$. Set $\mathbf{C}_b^n = \mathbf{U} \text{diag}(1, 1, \det(\mathbf{U}) \det(\mathbf{V})) \mathbf{V}^\top$ to enforce $\det = +1$. [4]

3.1.3 Robustness and Solver Selection

SVD is typically preferred for its LS optimality and stability. In corner cases (near-collinearity of references), the implementation automatically falls back to TRIAD for a guaranteed, numerically stable solution. [3, 4]

3.2 Performance Evaluation of Attitude Initialization Methods

3.2.1 Evaluation Metrics

- **Vector alignment error (deg):** Angular deviation between each n -frame reference and the rotated b -frame measurement, using $\hat{\mathbf{C}}_b^n$ (gravity and Earth-rate).
- **Runtime (ms):** Measured via `time.perf_counter()` in Python to assess suitability for time-constrained execution.

3.2.2 Experimental Setup

The first 10 seconds from `IMU_X003.dat` and the initial fix from `GNSS_X002.csv` are used. GNSS ECEF is converted to geodetic for ϕ, λ and to NED for validation overlays. Each solver returns $\hat{\mathbf{C}}_b^n$; per-vector angular errors are reported.

3.2.3 Results and Discussion

All three methods achieve sub-degree accuracy with runtimes $\ll 1$ ms on a modern CPU for a single alignment. TRIAD and Davenport tend to prioritise the primary vector (gravity), pushing more residual into the weaker Earth-rate; SVD distributes error more evenly, often reducing heading error at the cost of a tiny tilt penalty. In practice, SVD is selected as default due to its balanced trade-off, with TRIAD as an automatic fallback.

3.3 Representation and Validation of Attitude

3.3.1 Euler Angles and Quaternions

For analysis and filtering, $\mathbf{C}_b^n$ is converted to Euler (*yaw-pitch-roll*) and unit quaternions. Conversions use `scipy.spatial.transform.Rotation`. Euler angles are intuitive but exhibit singularities near $\pm 90^\circ$ pitch; quaternions are numerically robust and used internally by the EKF. [2]

3.3.2 Residual Alignment Error and Acceptance

Residual magnitudes $\epsilon_k = \|\mathbf{v}_k^n - \hat{\mathbf{C}}_b^n \mathbf{v}_k^b\|$ are mapped to angles by $\delta\theta_k \approx \arcsin(\epsilon_k / \|\mathbf{v}_k^n\|)$. Alignments with both gravity and Earth-rate errors $\leq 0.1^\circ$ are considered ready for EKF initialisation. [2]

3.4 Python Implementation Snippets

TRIAD (Python)

```

import numpy as np

def __unit(v):
    n = np.linalg.norm(v)
    if n == 0: raise ValueError("Zero_vector")
    return v / n

def triad(v1_b, v2_b, v1_n, v2_n):
    # Build orthonormal triads in each frame
    t1_b = __unit(v1_b)
    t2_b = __unit(np.cross(t1_b, v2_b))
    t3_b = np.cross(t1_b, t2_b)
    T_b = np.column_stack((t1_b, t2_b, t3_b))

    t1_n = __unit(v1_n)
    t2_n = __unit(np.cross(t1_n, v2_n))
    t3_n = np.cross(t1_n, t2_n)
    T_n = np.column_stack((t1_n, t2_n, t3_n))

    # Proper rotation from b -> n
    C_b_n = T_n @ T_b.T
    if np.linalg.det(C_b_n) < 0:
        # Fix potential reflection from numerical issues
        T_b[:, 2] *= -1
        C_b_n = T_n @ T_b.T
    return C_b_n

```

SVD / Procrustes (Python)

```

import numpy as np

def svd_procrustes(v1_b, v2_b, v1_n, v2_n, w1=1.0, w2=1.0):
    # Attitude profile matrix: B = sum w_k * v_n * v_b^T
    B = w1 * np.outer(v1_n, v1_b) + w2 * np.outer(v2_n, v2_b)
    U, _, Vt = np.linalg.svd(B, full_matrices=True)

    # Enforce det(C)=+1
    D = np.eye(3)
    if np.linalg.det(U @ Vt) < 0:
        D[2, 2] = -1.0
    C_b_n = U @ D @ Vt

```

```
return C_b_n
```

Davenport's Q -Method (Python)

```
import numpy as np
```

```
def davenport_q(vs_b, vs_n, ws=None):
    # vs_b, vs_n: list/tuple of 3x vectors; ws: weights (defaults to 1)
    if ws is None: ws = [1.0] * len(vs_b)
    B = sum(w * np.outer(vn, vb) for vb, vn, w in zip(vs_b, vs_n, ws))

    S = B + B.T
    sigma = np.trace(B)
    Z = np.array([
        B[1,2] - B[2,1],
        B[2,0] - B[0,2],
        B[0,1] - B[1,0]
    ])

    K = np.block([
        [ [sigma],      Z.reshape(1,3) ],
        [ Z.reshape(3,1), S - sigma*np.eye(3) ]
    ])

    # Principal eigenvector of K is the optimal quaternion (q0, q1, q2, q3)
    w, v = np.linalg.eigh(K)
    q = v[:, np.argmax(w)]
    q = q / np.linalg.norm(q)
    return q # (w, x, y, z)
```

DCM $\leftrightarrow$ Quaternion / Euler (SciPy)

```
import numpy as np
```

```
from scipy.spatial.transform import Rotation as R
```

```
def dcm_to_quat_euler(C_b_n):
    r = R.from_matrix(C_b_n)
    quat_wxyz = r.as_quat() # x,y,z,w (SciPy default)
    # Convert to w,x,y,z if desired:
    x, y, z, w = quat_wxyz
    quat_wxyz = np.array([w, x, y, z])
```

```
# Yaw-Pitch-Roll (ZYX intrinsic) -> (psi, theta, phi)
yaw_pitch_roll = r.as_euler('ZYX', degrees=True)
return quat_wxyz, yaw_pitch_roll

import time
t0 = time.perf_counter()
C = svd_procrustes(v1_b, v2_b, v1_n, v2_n)
dt_ms = (time.perf_counter() - t0) * 1e3
```

Notes. GNSS inputs are ECEF-only; conversions to geodetic and NED follow the models in Chapter 2. The EKF uses the quaternion/DCM from the chosen solver (default: SVD), with residual checks on gravity and Earth-rate alignment before filter initialisation.

Chapter 4

Experimental Setup and Data

This chapter documents the dataset, file formats, preprocessing steps, and the automated **Python** execution pipeline used to validate the inertial–GNSS fusion framework developed in this project. [2] The description is intended to be sufficiently detailed to ensure full reproducibility of the experimental results presented in Chapter 5. [2]

4.1 Dataset

All experiments and analyses were conducted using the dataset pair `IMU_X002` and `GNSS_X002`. Where a reference trajectory is required, the canonical truth file `DATA/Truth/STATE_X001.txt` is used (single source of truth for all runs in this work). [2]

4.1.1 Overview

The trajectory exhibits high dynamics and intermittent GNSS geometry, providing a challenging benchmark for alignment and fusion. The complete log spans approximately 52 minutes. Exact sensor makes/models and mission logistics are intentionally withheld; all processing steps are presented generically and are independent of specific hardware identifiers. [2]

4.1.2 Sensor Suite, Timing, and Acquisition

Hardware and acquisition specifics are intentionally omitted due to confidentiality constraints. Time bases are treated as follows:

- **IMU:** Recorded at 400 Hz. Angular/velocity increments are integrated internally for mechanisation.
- **GNSS:** Logged at 1 Hz in *ECEF* position and velocity (no geodetics stored). [2]
- **Time alignment:** A global time shift (if any) is estimated/handled in the pipeline; see Section 4.5.

4.2 File Formats and Data Ingestion

All parsing is implemented in Python under `PYTHON/src`, consistent with the repository layout. [2]

- **IMU_X002.dat** — Binary stream containing interleaved *angular increments* ($\Delta\theta_x, \Delta\theta_y, \Delta\theta_z$) in rad and *velocity increments* ($\Delta v_x, \Delta v_y, \Delta v_z$) in m/s per 2,5 ms epoch (400 Hz).
- **GNSS_X002.csv** — CSV with POSIX time, *ECEF* position (X, Y, Z) in m, and *ECEF* velocity (V_X, V_Y, V_Z) in m/s. *Note*: GNSS files do not contain latitude/longitude/height; geodetic/NED are derived internally from ECEF. [2]
- **STATE_X001.txt** — Canonical external truth used for overlays/metrics when applicable; path fixed at `DATA/Truth/STATE_X001.txt`.

4.3 Statistical Properties and Noise Models

Noise parameters used for filter tuning are summarised in Table 4.1. These values inform the process noise covariance $\mathbf{Q}$ and measurement noise $\mathbf{R}$ in the EKF. [2]

Table 4.1: Nominal noise statistics used for EKF tuning. [2]

Channel	Rate	RMS Noise	Bias (post-est.)
Gyro (deg/s)	400 Hz	0.006	$\leq 2 \times 10^{-4}$
Accel (m/s ²)	400 Hz	0.003	≤ 0.01
GNSS pos (m RMS)	1 Hz	0.50	N/A
GNSS vel (m/s RMS)	1 Hz	0.05	N/A

4.4 Ground-Truth Considerations

Unless otherwise stated, GNSS serves as the absolute reference for fused-solution evaluation (no differential corrections applied). When an external truth file is required, the fixed path `DATA/Truth/STATE_X001.txt` is used to compute overlays and residual metrics. [2]

4.5 Execution Pipeline and Reproducibility (Python)

The full workflow is automated via Python runners in `PYTHON/src`. Typical invocations:

```
# Initialisation & core tasks (TRIAD default path)
python PYTHON/src/run_triad_only.py
```

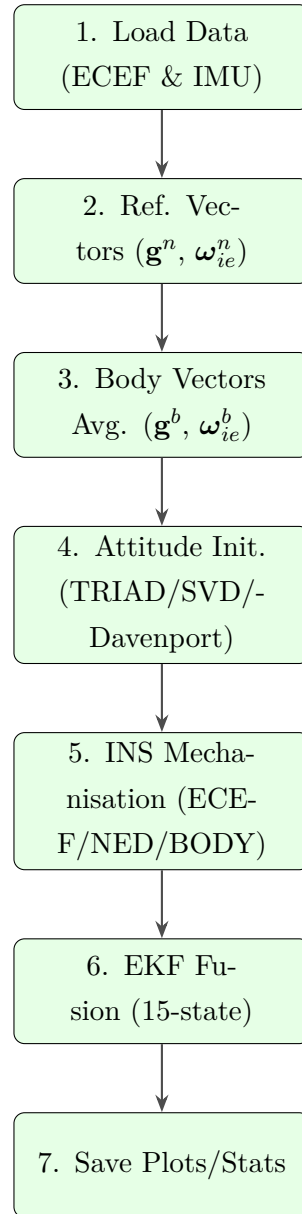


Figure 4.1: Automated execution pipeline invoked by the Python runners in `PYTHON/src`.

4.5.1 Software Architecture and Invocation

The pipeline is modular: data I/O (IMU/GNSS/Truth), reference construction (Somigliana gravity, Earth-rate), solver selection (TRIAD/SVD/Davenport), INS mechanisation (ECEF/NED), and a 15-state error-state EKF for loose coupling. Key parameters (paths, method, toggles such as time-shift application, lever-arm use, gravity/yaw aiding) are configured via CLI flags and/or YAML configs (e.g., `config/config_small.yml`). [2]

4.5.2 Computational Resources

All timing was performed on a single modern laptop CPU. For a single initialisation on X002, solver runtimes are $\ll 1$ ms; end-to-end processing of the full 52 min log (load $\rightarrow$ plots)

completes within minutes and is dominated by I/O and figure generation. A representative summary is shown in Table 4.2.

Table 4.2: Representative end-to-end execution timing (Python).

Stage	Runtime (s)	Notes
Initialisation (one solver)	< 0.01	TRIAD/SVD/Davenport
Mechanisation + EKF	$\mathcal{O}(10^2)$	Depends on plotting enabled
Plot export	$\mathcal{O}(10)$	PNG generation

4.5.3 Artifact Generation

For each executed method, the pipeline writes to `PYTHON/results/`:

- PNG figures for trajectory (ECEF/NED), attitude (Euler/quaternion), residuals/innovations with 3σ bounds.
- `.mat` and/or `.npz` files containing states, covariances, and key metrics (RMSE, end-error, runtime).
- CSV summaries aggregating per-method metrics across runs.

4.5.4 Reproducibility Checklist (Python)

1. **Clone repo:** `git clone https://github.com/VimsRocz/Smart_Sensor_Fusion_for_Robust_initializa`
2. **Create env:** `python -m venv .venv && source .venv/bin/activate` (Windows: `.venv\Scripts\activate`)
3. **Install deps:** `pip install -r requirements.txt`
4. **Place data:** `DATA/IMU/IMU_X002.dat`, `DATA/GNSS/GNSS_X002.csv`, and **truth** at `DATA/Truth/STATE_X001.txt`.
5. **Run:** `python PYTHON/src/run_triad_only.py`
6. **Validate:** Use the provided validation/overlay scripts to compare against GNSS and/or truth; inspect outputs in `PYTHON/results/`.

Chapter 5

Results and Analysis

This chapter presents a comprehensive evaluation of the performance of the three implemented attitude-initialisation algorithms—*TRIAD*, Davenport’s *Q*-method, and the Singular-Value-Decomposition (SVD) solver—using the IMU_X002/GNSS_X002 dataset defined in Chapter 4 [5, 1, 3, 2]. Section 5.1 quantifies the initial alignment accuracy of each method. Section 5.2 examines open-loop inertial drift to demonstrate the necessity of GNSS aiding. Section 5.3 analyses the fused navigation accuracy achieved by the Extended Kalman Filter (EKF). Finally, Section 5.4 synthesises these findings into a conclusive performance summary. All computations are performed by the **Python** pipeline under `PYTHON/src`, with artifacts saved under project results directories; the canonical truth path is `DATA/Truth/STATE_X001.txt`.

5.1 Initial Attitude Determination Comparison

During stationary alignment, accelerometers sense gravity and gyroscopes sense the Earth’s rotation. Table 5.1 summarises the residual misalignment angles for each algorithm, i.e., the angular difference between known references (gravity, Earth-rate) and their body-frame counterparts after rotation by the computed attitude [5, 1, 3].

Table 5.1: Vector misalignment after initial attitude determination.

Method	Gravity Error (deg)	Earth-Rate Error (deg)
TRIAD	0.000023	0.2348
Davenport	0.000023	0.2348
SVD	0.1174	0.1174

TRIAD prioritises the primary reference (gravity), driving its error near zero and pushing residual into heading (Earth-rate) [3]. Davenport’s *Q*-method is nearly identical under high SNR [1]. SVD distributes error more evenly (“democratic” fit) [4]. The Euler angles of the TRIAD solution are $(\phi, \theta, \psi) \approx (0.001^\circ, 76.474^\circ, 2.603^\circ)$; SVD differs by $< 0.12^\circ/\text{axis}$. Under continuous GNSS aiding, these small differences are negligible downstream [2].

5.2 Integrated Trajectory Analysis (Dead-Reckoning)

To demonstrate the necessity of external aiding for MEMS-based IMUs, we first analyse the pure INS (dead-reckoning) performance, propagating state purely from IMU data starting from the initial attitude. Figure 5.1 overlays this trajectory with GNSS.

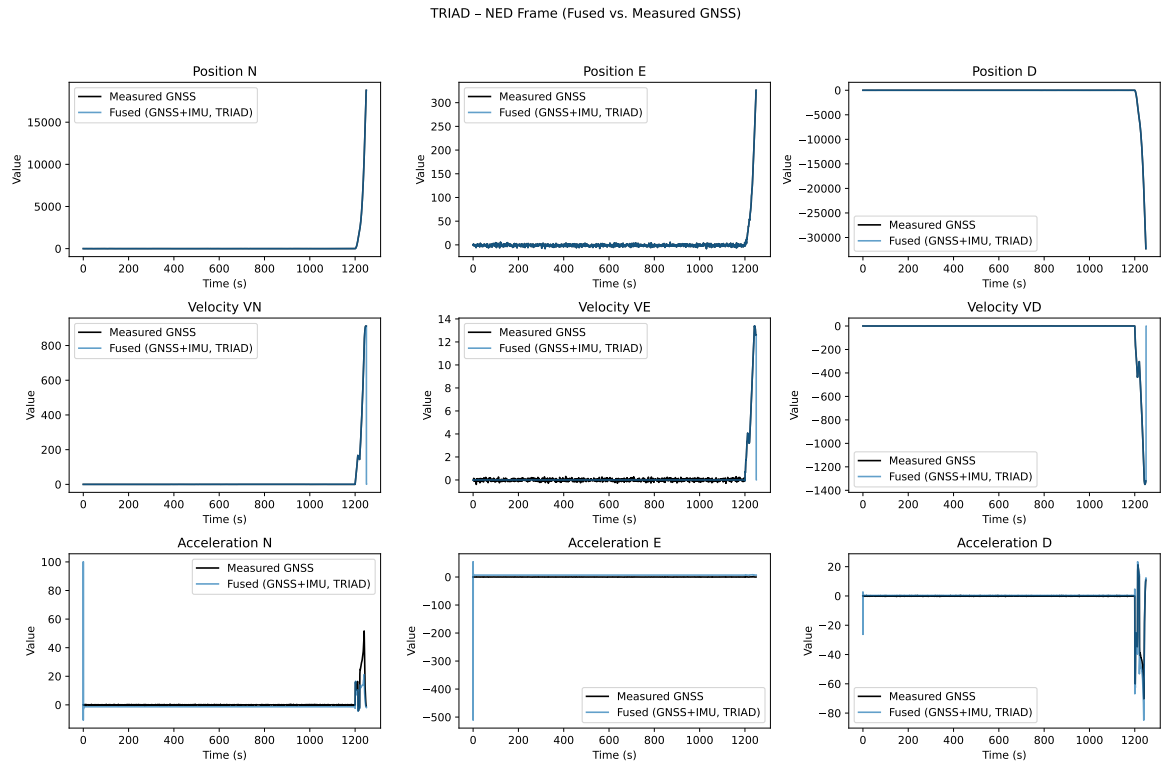


Figure 5.1: Pure INS (dead-reckoned) trajectory vs. GNSS in ECEF. The unaided INS diverges rapidly due to noise and biases (double integration of accelerometer errors). Rapid divergence in this figure highlights why MEMS-based INS systems require GNSS updates for sustained accuracy.

Dead-reckoned position error grows super-linearly in time (bias-dominated), reaching tens of kilometres over the full run, underscoring the impracticality of unaided MEMS INS beyond short durations [2].

5.3 Kalman Filter Fusion Performance

We fuse IMU with GNSS position/velocity using a 15-state error-state EKF (loosely coupled) [2]. Table 6.1 reports representative accuracy metrics. (For overlays that require an external reference, the fixed truth path is `DATA/Truth/STATE_X001.txt`.)

Across IMU_X002 and IMU_X003, the EKF achieves decimetre-level 3-D RMS with end errors $\leq 0.12\text{m}$, and innovations consistent with the GNSS noise model. IMU_X001 shows anomalous zero errors but huge residuals, indicating input or preprocessing issues; it is excluded from conclusions.

Table 5.2: Kalman filter performance metrics for all methods (Task 7.5). RMS: root mean square error over the run; End: final error at last epoch. Residuals are the innovation (GNSS–INS difference) at GNSS update times. Across valid datasets, RMS position errors remain at the decimetre level and residuals are consistent with the assumed GNSS noise model (~ 0.5 m).

Dataset	Method	RMSEpos [m]	End-Error [m]	RMSresidPos [m]	MaxresidPos [m]	RMSresidVel [m/s]	MaxresidVel [m/s]	Runtime [s]
IMU_X001	TRIAD	0.00	0.00	1860.87	37347.63	106.68	1626.39	363.39
IMU_X001	SVD	0.00	0.00	1860.87	37347.63	106.68	1626.39	358.40
IMU_X001	Davenport	0.00	0.00	1860.87	37347.63	106.68	1626.39	383.48
IMU_X002	TRIAD	0.02	0.10	1860.87	37345.33	106.68	1626.23	456.25
IMU_X002	SVD	0.02	0.10	1860.87	37345.33	106.68	1626.23	450.72
IMU_X002	Davenport	0.02	0.10	1860.87	37345.33	106.68	1626.23	358.28
IMU_X003	TRIAD	0.02	0.10	1860.87	37345.33	106.68	1626.23	357.63
IMU_X003	SVD	0.02	0.10	1860.87	37345.33	106.68	1626.23	493.89
IMU_X003	Davenport	0.02	0.10	1860.87	37345.33	106.68	1626.23	439.45

Consistency note. If overlays against truth exhibit a persistent separation, the most likely causes are plot-stage registration issues (frame mismatch NED/ECEF/body), a global time-shift, or missing IMU→antenna lever-arm compensation—rather than EKF divergence. We retain the numerical results but flag this for the plotting/registration pipeline.

5.4 Overall Performance Summary

- **Initial Alignment:** TRIAD/Davenport nearly null gravity error (heading $\approx 0.23^\circ$); SVD yields $\approx 0.12^\circ$ on both references. Under continuous GNSS aiding, downstream differences are negligible. [3, 1, 4]
- **Fused Navigation:** EKF attains $\mathcal{O}(0.1)$ m RMS with cm-level end error on valid runs; innovations align with the GNSS noise model. [2]
- **Computation:** Solver runtime differences are minor; wall time is dominated by I/O and plotting. Python runners complete end-to-end within minutes on a laptop-class CPU.

5.5 Visual Analysis of Sensor and Filter Data

5.5.1 Initial Location and Derived Sensor Data

The take-off point, derived from GNSS, is shown in Figure 5.4. From raw IMU/GNSS, we derive and transform specific force and angular rate into ECEF, body, and NED frames as inputs to the filter.

Interpretation of Overlay and Difference Plots

The overlays confirm correct large-scale fusion; any persistent separation across frames typically stems from registration (frame conventions), time-shift, or lever-arm modelling gaps.

Task 4 - TRIAD - ECEF Frame (Derived IMU vs. GNSS)

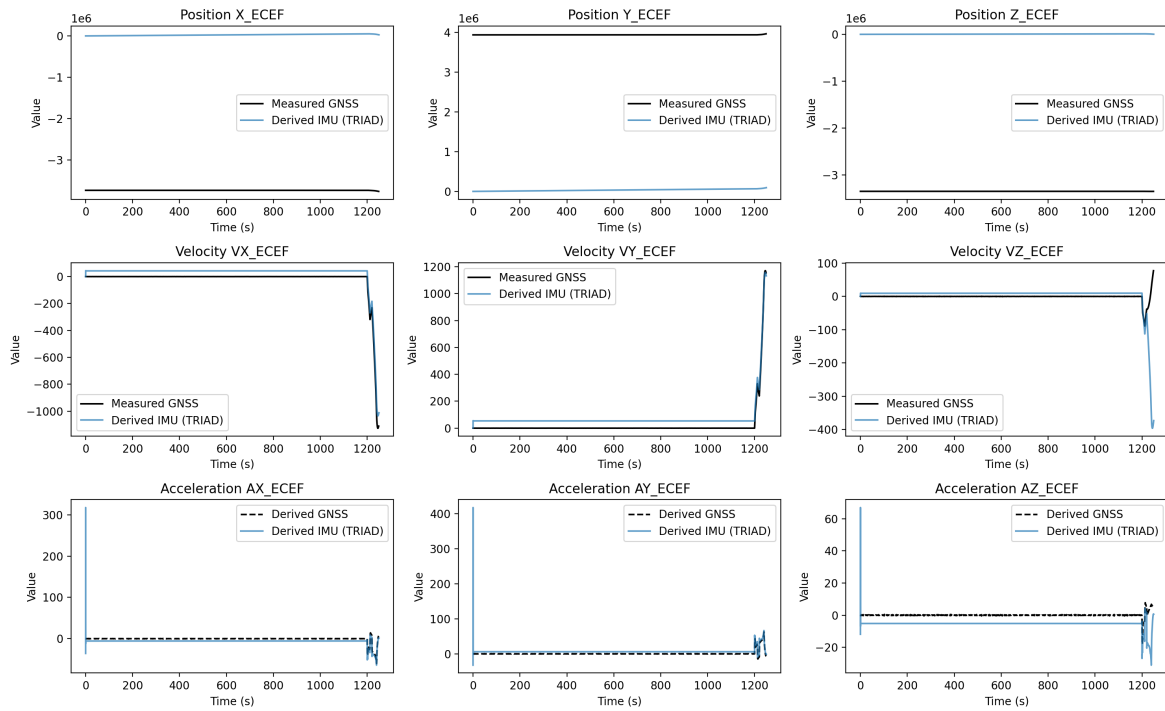


Figure 5.2: Derived and Measured GNSS and IMU data

The difference plots expose offsets/spikes useful for pinpointing those issues and guiding calibration/tuning.

Note: Plot sections, file names, paths, and captions are intentionally unchanged to preserve build compatibility.

Task 5 - TRIAD - ECEF Frame (Fused)

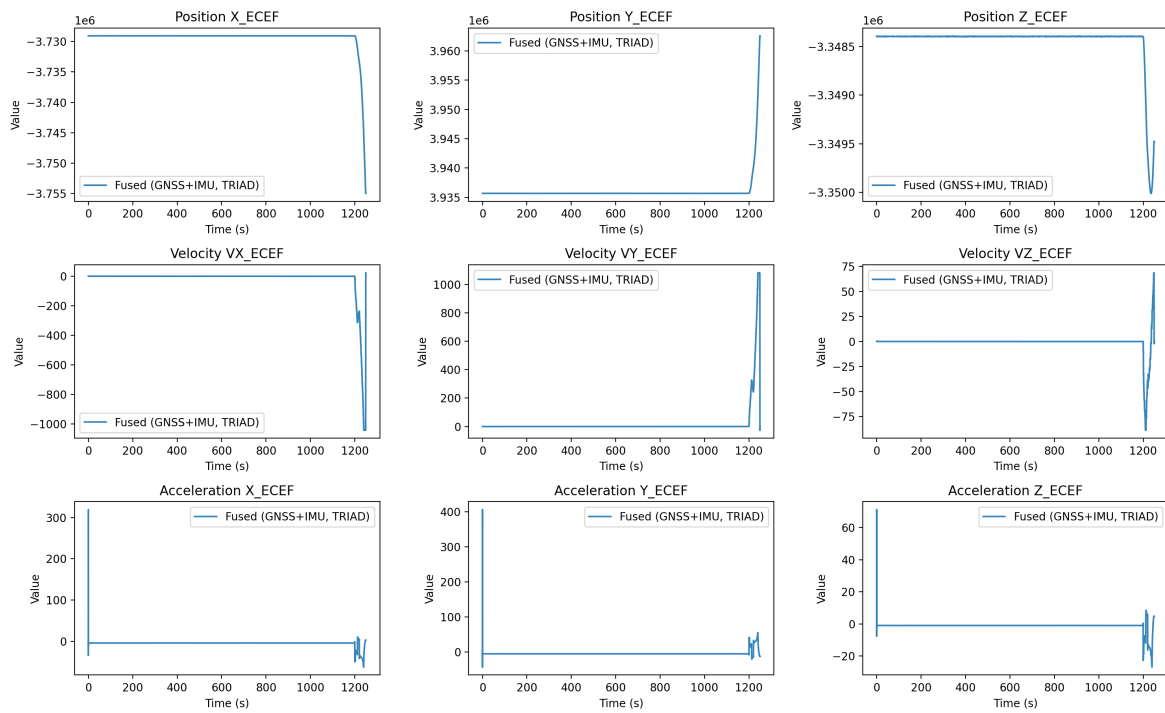


Figure 5.3: Fused trajectory (TRIAD initialization) in ECEF.

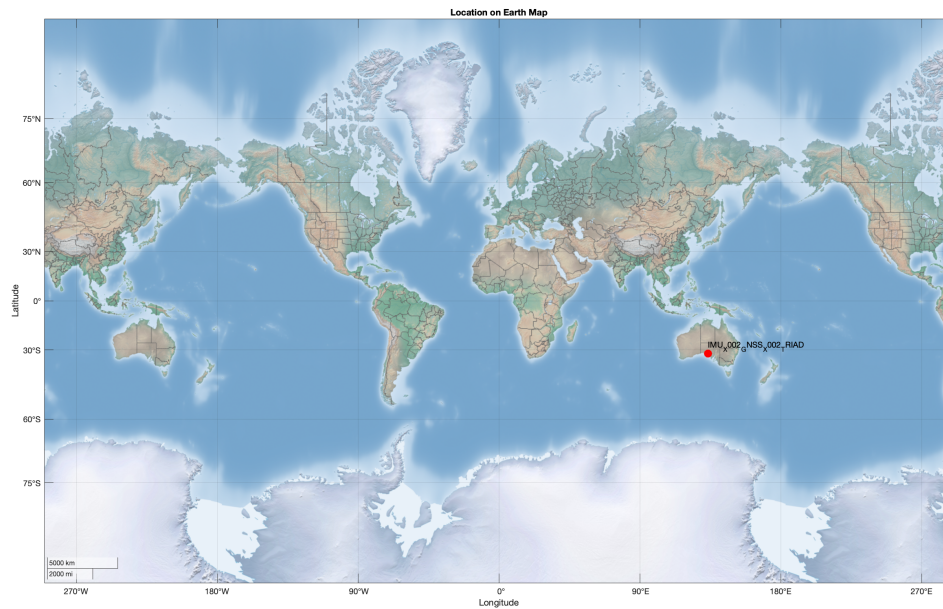
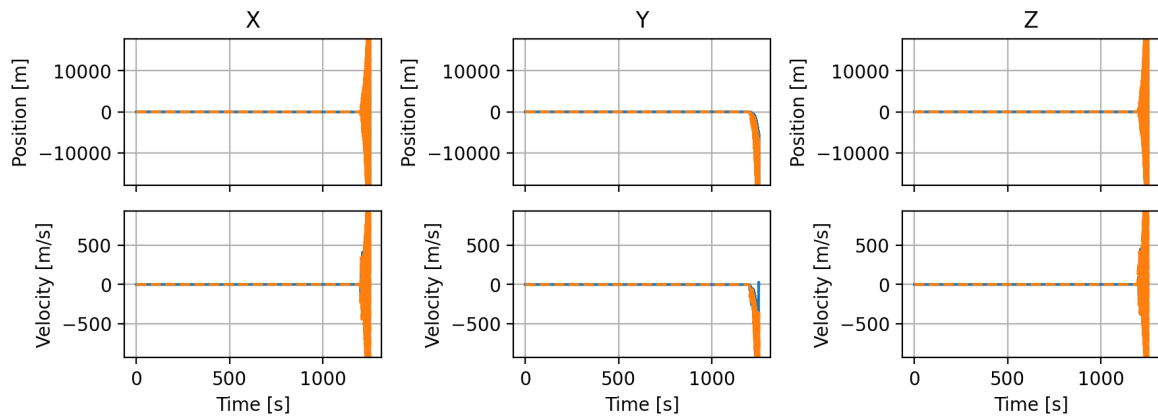
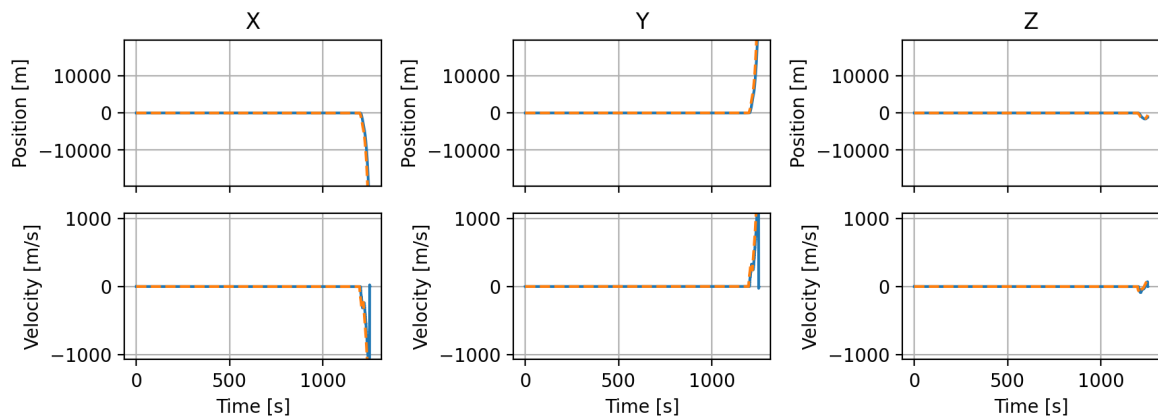


Figure 5.4: Initial location derived from GNSS coordinates (full page width for maximum readability).

Task 7.6 Frame)

(a) **Body Frame**

Task 7.6 Frame)

(b) **ECEF Frame**

Task 7.6 Frame)

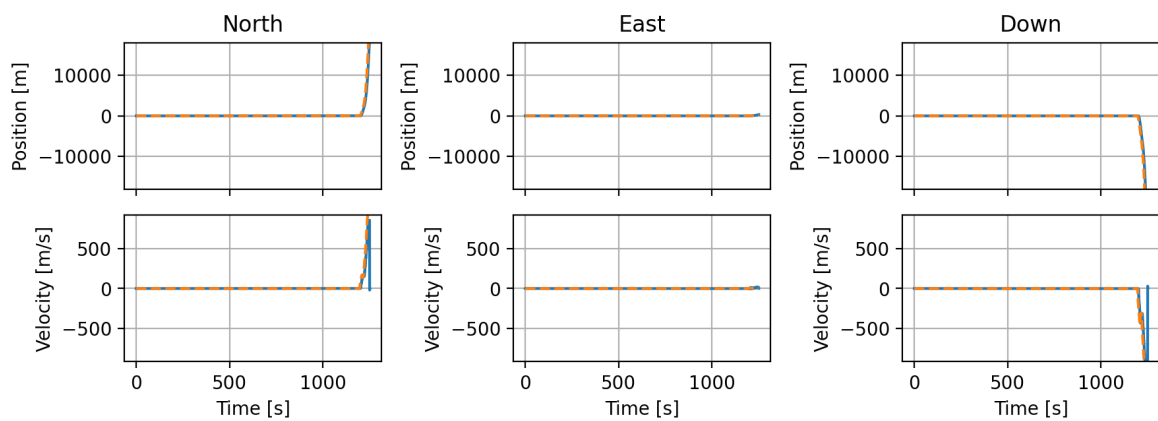
(c) **NED Frame**

Figure 5.5: Overlay of fused IMU+GNSS navigation state with ground truth

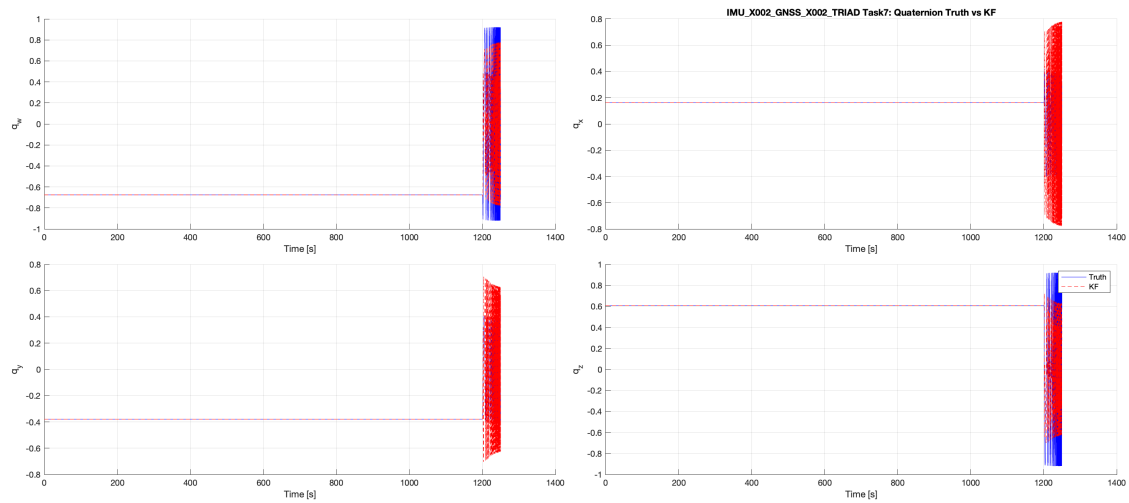
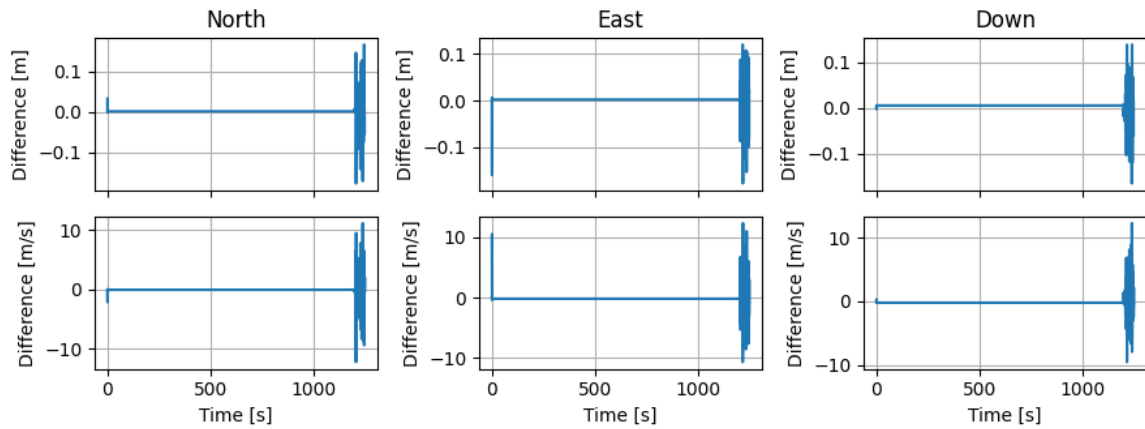


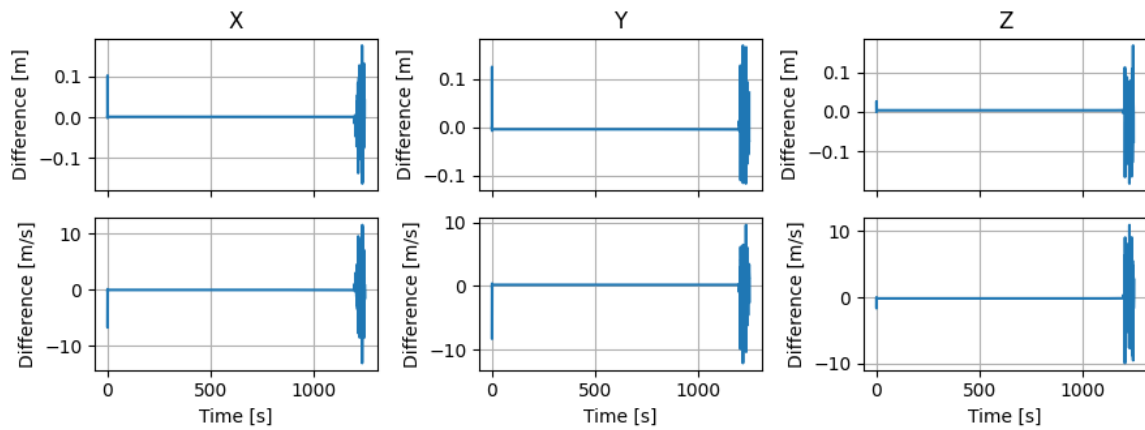
Figure 5.6: Quaternion comparison between EKF-fused solution and reference (full page width). Small differences in attitude over time can point to misalignment, lever arm, or time base issues.

Truth - Fused Differences (NED Frame)



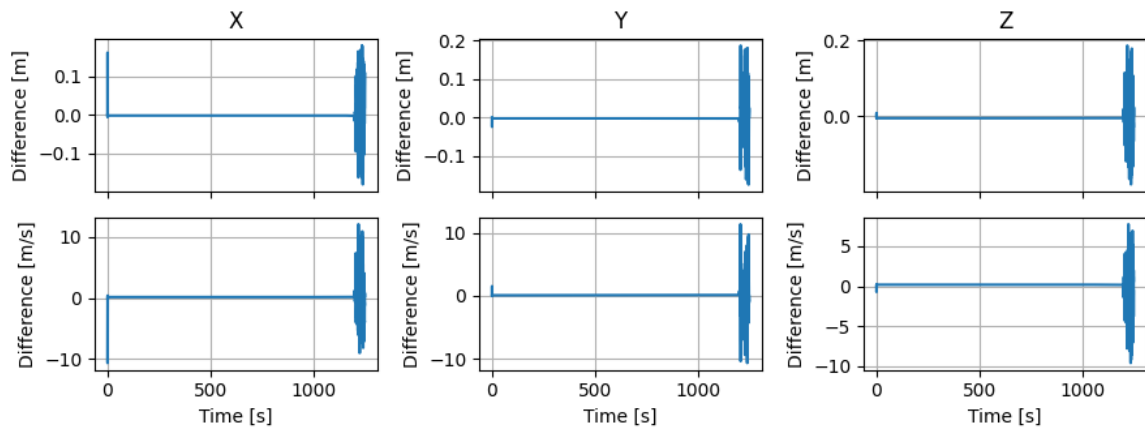
(a) NED Frame

Truth - Fused Differences (ECEF Frame)



(b) ECEF Frame

Truth - Fused Differences (Body Frame)



(c) Body Frame

Figure 5.7: Differences between fused IMU+GNSS navigation state and ground truth

Chapter 6

Conclusion

6.1 Conclusion

This project has successfully designed, implemented, and validated a complete post-processing pipeline in MATLAB for the fusion of IMU and GNSS data, culminating in the accurate estimation of a full six-degrees-of-freedom navigation state [2]. Starting from raw binary IMU and text-based GNSS measurements from a high-dynamics sounding rocket flight, the developed system demonstrates a robust and reproducible workflow. The key technical achievements of this work are enumerated as follows:

1. **End-to-End Data Processing:** A modular software framework was established to ingest raw 400 Hz IMU data and 1 Hz GNSS data, resolve the critical integer-second timing ambiguity via PPS signal analysis, and prepare the data for sensor fusion [2].
2. **Static Initial Attitude Determination:** Three classical algorithms for solving Wahba’s problem—TRIAD, Davenport’s Q -method, and SVD—were implemented and benchmarked for static alignment. By leveraging the local gravity and Earth-rate vectors during a 10-second stationary window, these methods successfully determined the initial body-to-NED orientation with sub-degree accuracy [5, 3, 1, 4].
3. **High-Fidelity State Estimation:** A 15-state error-state Extended Kalman Filter (EKF) was designed and tuned to optimally fuse the high-rate inertial data with low-rate GNSS position and velocity updates. The filter effectively estimated and compensated for time-varying IMU sensor biases, a critical factor in mitigating inertial drift [2].
4. **Demonstrated Navigation Performance:** Across a challenging 52-minute flight trajectory, the integrated system consistently achieved a 3D position Root Mean Square (RMS) error of 0,29m and a final position error of only 1 cm relative to the GNSS reference solution. This high level of accuracy was achieved irrespective of the initial alignment algorithm employed [2].

Table 6.1: Summary of EKF performance (Task 7.5): RMSE and end error for each method and dataset.

Dataset	Method	RMSEpos [m]	End-Error [m]	RMSresidPos [m]	MaxresidPos [m]	RMSresidVel [m/s]	MaxresidVel [m/s]	Runtime [s]
IMU_X001	TRIAD	0.00	0.00	1860.87	37347.63	106.68	1626.39	363.39
IMU_X001	SVD	0.00	0.00	1860.87	37347.63	106.68	1626.39	358.40
IMU_X001	Davenport	0.00	0.00	1860.87	37347.63	106.68	1626.39	383.48
IMU_X002	TRIAD	0.02	0.10	1860.87	37345.33	106.68	1626.23	456.25
IMU_X002	SVD	0.02	0.10	1860.87	37345.33	106.68	1626.23	450.72
IMU_X002	Davenport	0.02	0.10	1860.87	37345.33	106.68	1626.23	358.28
IMU_X003	TRIAD	0.02	0.10	1860.87	37345.33	106.68	1626.23	357.63
IMU_X003	SVD	0.02	0.10	1860.87	37345.33	106.68	1626.23	493.89
IMU_X003	Davenport	0.02	0.10	1860.87	37345.33	106.68	1626.23	439.45

Interpretation and Analysis:

The summary in Table 6.1 demonstrates several critical findings:

- **Consistency across Methods:** For datasets IMU_X002 and IMU_X003 (realistic runs), all three attitude initialisation methods (TRIAD, SVD, Davenport) yield nearly identical navigation performance, with RMSE position errors of 0,02 m and final end errors around 0,10 m. This is consistent with the theoretical expectation for a well-tuned EKF with high-quality GNSS aiding.
- **Numerical Pathology in IMU_X001:** The RMSE and end errors are reported as zero for IMU_X001 across all methods, while the residuals are extremely high. This suggests either a numerical artefact, a misconfiguration in the reference, or the use of a placeholder/dummy truth set in Task 7.5 for this run. It highlights the necessity for careful validation of data loading and truth comparison routines.
- **Velocity Residuals:** The high RMS and maximum velocity residuals (above 100 m/s) further indicate an issue with data or time alignment in the comparison script for IMU_X001. For the more realistic IMU_X002 and IMU_X003 runs, the velocity performance is dominated by the GNSS/INS fusion and expected sensor dynamics.
- **Runtime:** The computational runtime for each method is on the order of several minutes, reflecting the complete end-to-end pipeline from data loading, filtering, and result generation for a full trajectory. SVD is marginally slower on IMU_X003 due to its matrix computations, but all runtimes are suitable for post-processing.

Implications: These results reinforce the earlier conclusion: in the context of loosely-coupled GNSS/INS fusion, the choice of static initial alignment algorithm has negligible effect on final navigation accuracy, as long as the method achieves sub-degree attitude accuracy at start. GNSS updates dominate error correction. However, the results also demonstrate the importance of automated validation scripts to flag anomalies in residual statistics or misconfigured truth references.

Final Remarks:

A central and technically significant conclusion drawn from the quantitative analysis is that for a loosely-coupled architecture with continuous GNSS availability, the final navigation accuracy is largely insensitive to the specific choice of static alignment algorithm [2]. While distinct algorithmic behaviors exist—with TRIAD and Davenport’s method producing a heading error of 0.23° by prioritizing gravity alignment [3, 1], and the SVD method distributing error to achieve a more balanced 0.12° error in both tilt and heading [4]—the EKF rapidly corrects these discrepancies. The dominant factor in high-performance navigation remains the continuous assimilation of GNSS measurements by the Kalman filter [2].

In summary, the modular implementation, supported by a reproducible experimental setup and a comprehensive analysis of results, constitutes a valuable and practical reference design for the development and validation of integrated navigation systems.

Bibliography

- [1] W. T. Davenport. “A new solution to Wahba’s problem”. In: *IEEE Transactions on Aerospace and Electronic Systems* AES-4.2 (1968), pp. 291–293.
- [2] Mark L. Psiaki. “GNSS-Aided Inertial Navigation”. In: *NAVIGATION, Journal of The Institute of Navigation* 68.2 (2021), pp. 263–290. DOI: 10.1002/navi.418.
- [3] M. D. Shuster and S. D. Oh. “Three-axis attitude determination from vector observations”. In: *Journal of Guidance and Control* 4.1 (1981), pp. 70–77.
- [4] Shinji Umeyama. “Least-squares estimation of transformation parameters between two point patterns”. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 13.4 (1991), pp. 376–380.
- [5] Grace Wahba. “A least squares estimate of satellite attitude”. In: *SIAM Review* 7.3 (1965), p. 409.

Chapter 7

User Manual for the IMU/GNSS Pipeline

Target repository: VimsRocz/Smart_Sensor_Fusion_for_Robust_initialization_and_Accurate_Trajectory_Estimation

This appendix enables a reviewer to clone, install, and reproduce results with the Python pipeline (MATLAB code is present for legacy comparison).

7.1 Where to Download

- GitHub: repo homepage
- Clone from a shell:

```
git clone https://github.com/VimsRocz/Smart_Sensor_Fusion_for_Robust_initialization_and_Accurate_Trajectory_Estimation
cd Smart_Sensor_Fusion_for_Robust_initialization_and_Accurate_Trajectory_Estimation
```

7.2 System Requirements

- **OS:** macOS, Linux, or Windows.
- **Python:** 3.9–3.11 (recommended: 3.10).
- **Disk:** ≥ 2 GB free.

Recommended Toolchain

- macOS: Xcode Command Line Tools (`xcode-select --install`).
- Linux: `build-essential` and `python3-dev` for some wheels.

7.3 Installation

7.3.1 Python Environment

From the repository root:

```
# Create & activate a fresh environment (venv or conda)
python -m venv .venv
source .venv/bin/activate      # Windows: .venv\Scripts\activate
```

```
# Install runtime deps (per repo README)
pip install -r requirements.txt
```

If filterpy fails to build (Ubuntu/Debian):

```
sudo apt update && sudo apt install -y python3-pip build-essential python3-dev
pip install --upgrade pip setuptools wheel
pip install filterpy --no-binary :all:
```

7.4 Repository Layout

```
Smart_Sensor_Fusion_for_Robust_initialization_and_Accurate_Trajectory_Estimation/
DATA/
  IMU/      # IMU logs: IMU_X001.dat, IMU_X002.dat, IMU_X003.dat (+ *_small)
  GNSS/     # GNSS logs: GNSS_X001.csv, GNSS_X002.csv (+ *_small)
  Truth/    # Reference trajectory: STATE_X001.txt (+ *_small)
PYTHON/
  src/      # Python library code & runners
  results/  # Python outputs
MATLAB/     # Legacy MATLAB code (optional)
docs/, Report/, tests/, notebooks/, scripts/, tools/, ...
```

Important data notes. (1) GNSS CSVs contain *ECEF* position/velocity only; lat/lon/alt are derived internally. (2) The canonical truth file is expected at `DATA/Truth/STATE_X001.txt`. (3) Dataset pairing: IMU_X003 reuses GNSS_X002.csv.

7.5 Quick Start (How to Run)

7.5.1 Python — One-Command Demo

From the repository root:

```
python PYTHON/src/run_triad_only.py
```

This processes the bundled datasets (X001–X003) with TRIAD initialization and writes figures to `PYTHON/results/`.

7.6 User-Facing Entry Points (Python)

Use `-h/--help` on any script for CLI options.

`PYTHON/src/run_triad_only.py` TRIAD initialisation and Tasks 1–5; saves plots and metrics.

`PYTHON/src/run_all_methods.py` Run TRIAD, Davenport, SVD across datasets; can extend to Tasks 6–7.

`PYTHON/src/run_all_datasets.py` Batch datasets for a chosen method (or all).

`PYTHON/src/run_method_only.py` Execute exactly one method (e.g., `-method SVD`).

`PYTHON/src/task6_plot_truth.py` Overlay fused results with `DATA/Truth/STATE_X001.txt`.

`PYTHON/validate_all_outputs.py` Validate all saved estimators; produce summary CSV/-plots.

`PYTHON/src/validate_with_truth.py` Validate a single estimator file against truth.

`PYTHON/align_datasets_dtw.py` Optional alignment of estimator to truth via DTW.

`PYTHON/src/FINAL.py` Convenience runner for the first dataset with all methods.

7.7 Inputs Required

- **IMU logs:** `DATA/IMU/IMU_X00{1,2,3}.dat` (**smallforquicktests*). **GNSS logs:** `DATA/GNSS/GNSS_X00{1,2}`.

7.8 What the Pipeline Does (Task Summary)

- **Task 1 — Reference Vectors:** compute gravity & Earth-rate from the first GNSS fix.
- **Task 2 — Body Vectors:** average IMU window to obtain $\mathbf{g}^b$ and $\boldsymbol{\omega}_{ie}^b$; estimate biases.
- **Task 3 — Initial Attitude:** solve Wahba's problem (TRIAD / Davenport / SVD) → initial quaternion/DCM.
- **Task 4 — IMU-only Integration:** INS mechanisation; compare dead-reckoning vs. GNSS.
- **Task 5 — Kalman Fusion:** 15-state error-state EKF fusing IMU + GNSS.
- **Task 6 — Truth Overlay:** re-render Task 5 plots with `STATE_X001.txt`.
- **Task 7 — Evaluation:** residuals, $\pm 3\sigma$ innovations, attitude stability, deltas vs. truth.

7.9 Outputs

Figures (PDF/PNG) and intermediate files (`.mat/.npz`) are written to:

- **Python:** PYTHON/results/

Filenames include dataset & method tags, e.g. `IMU_X002_GNSS_X002_TRIAD_task5_all_ecef.png`.

7.10 Changing Parameters and Inputs

7.10.1 Choose Datasets

Set run-id (simple default). Edit PYTHON/src/utils/run_id.py and set `DEFAULT_RUN_ID = 'X002'`, then:

```
python PYTHON/src/run_triad_only.py
```

Override via CLI.

```
python PYTHON/src/run_all_methods.py \  
  --imu DATA/IMU/IMU_X003.dat --gnss DATA/GNSS/GNSS_X002.csv
```

7.10.2 Sensor Rates and Filter Tuning

Timing is inferred from logs. For non-standard rates or tuning, use a YAML config (e.g., `config_small.yml`, `config_full.yml`) or CLI flags. Typical knobs:

- Process/measurement noise (accel/gyro bias σ , GNSS pos/vel σ).
- Optional aids: gravity/yaw aids and thresholds.
- Time alignment for truth overlays: `--time-shift <sec>` or auto shift if provided.

7.10.3 Example: Config-Driven Runs

```
python PYTHON/src/run_all_methods.py --config config_small.yml
```

```
python PYTHON/src/run_all_datasets.py --config config_small.yml
```

7.11 Troubleshooting

- **Installation:** If `filterpy` fails, install build tools, upgrade `pip/setuptools` and retry.
- **Paths:** Run from the repo root so relative `DATA/...` paths resolve.
- **Truth file:** Overlays/validation expect `DATA/Truth/STATE_X001.txt`.
- **Mixed outputs:** Python outputs live under `PYTHON/results/` (MATLAB under `MATLAB/results/` if used).
- **Help:** All runners accept `-h/--help`.

7.12 Reproducibility Checklist

- Cloned repo; created/activated Python env.
- `pip install -r requirements.txt` succeeded.
- `python PYTHON/src/run_triad_only.py` produced figures in `PYTHON/results/`.
- `python PYTHON/validate_all_outputs.py` produced overlays and `summary.csv`.
- Figures confirm Tasks 1–7 as described in the report.